Software Requirements Specification (SRS)

TokCost: A Cross-Platform Token and Cost Estimation Framework for Large Language Models

Version 1.0

Prepared for: Final Year B.E. Major Project

Date: August 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for TokCost, a web-based platform that estimates token usage and cost for user prompts and files across multiple large language model (LLM) providers, namely ChatGPT, Claude, and Gemini. This document is intended to give any reader — including an AI assistant — full context of the system's purpose, scope, features, and constraints, so that development, review, or further assistance can proceed with a shared understanding of the project.

1.2 Scope

TokCost allows a user to submit a text prompt along with optional file attachments (PDF, DOCX, code files, images). The system parses the input, calculates token counts using each provider's official tokenizer, estimates cost for free and paid tiers, compresses the prompt to reduce token usage, recommends the most suitable platform, provides direct links to each platform, and generates a short summary of the session. The system does not process video or audio files, does not automatically submit prompts to any AI platform on the user's behalf, and does not fetch free-tier usage limits live (these are maintained as manually updated reference data).

1.3 Intended Audience

- Project team members (developers building the system)
- Project guide / evaluators reviewing the system design
- Any AI coding assistant (e.g. Claude, ChatGPT, GitHub Copilot) assisting with development, so it has full project context

1.4 Definitions and Abbreviations

Term	Definition
LLM	Large Language Model (e.g. GPT, Claude, Gemini)
Token	The unit of text an LLM uses internally to process input/output; pricing and limits are based on token count
Tokenizer	The algorithm/library a provider uses to convert text into tokens
API	Application Programming Interface — used to call each provider's tokenizer/service
SRS	Software Requirements Specification

2. Overall Description

2.1 Product Perspective

TokCost is a new, standalone web application. It is not a modification of an existing product. It interacts with three external services (OpenAI, Anthropic, Google) via their official APIs/SDKs for token counting and optionally for prompt compression and chat summarization.

2.2 Product Features (Summary)

- Accepts a text prompt and file attachments (PDF, DOCX, code, images)
- Parses files into raw text / extracts image dimensions
- Calculates token counts per provider using official tokenizers

- Estimates cost per provider (input + output, free and paid tiers)
- Compresses the prompt to reduce token usage while preserving meaning
- Recommends the most suitable platform based on cost and task type
- Displays free-tier/paid-tier reference limits and direct platform links
- Generates a short summary of each session
- Stores session history in a database

2.3 User Classes and Characteristics

User Class	Description
General User	Students, developers, professionals who want to estimate token/cost before using an AI platform. Needs no technical knowledge.
Administrator (future scope)	Would maintain the pricing configuration and free-tier reference data.

2.4 Operating Environment

- Client side: any modern web browser (Chrome, Edge, Firefox) on desktop or mobile
- Server side: Python 3.10+ runtime with FastAPI, hosted on a cloud platform (e.g. Render/Railway)
- Frontend hosted separately (e.g. Vercel/Netlify)
- Requires active internet connection for API calls to OpenAI, Anthropic, and Google

2.5 Design and Implementation Constraints

- Free-tier and paid-tier usage limits are not exposed via any public API; they are stored as manually updated static reference data
- Image token counts are calculated using each provider's published formula, since no universal standard exists
- Output token counts are estimates only, since the AI's actual response is not generated in advance
- API usage is subject to each provider's own rate limits and pricing, which may change over time

2.6 Assumptions and Dependencies

- Valid API keys for OpenAI, Anthropic, and Google Generative AI are available and configured as environment variables
- Users have a stable internet connection while using the platform
- Uploaded files are in supported formats and are not corrupted

3. Functional Requirements

ID	Requirement
FR-1	The system shall allow a user to enter a text prompt via a web form.
FR-2	The system shall allow a user to upload files in PDF, DOCX, code (.py, .java, .js, etc.), and image formats.
FR-3	The system shall parse uploaded files to extract raw text content or image dimensions.
FR-4	The system shall calculate the token count of the prompt and files for ChatGPT, Claude, and Gemini using each provider's official tokenizer/API.

FR-5	The system shall calculate the estimated cost of the input (and estimated output) on each platform, using a maintained pricing configuration.
FR-6	The system shall provide a compressed version of the user's prompt to reduce token usage while preserving meaning, and display the before/after token count.
FR-7	The system shall recommend the most suitable platform based on cost and task type.
FR-8	The system shall display each platform's known free-tier and paid-tier usage limits as reference information.
FR-9	The system shall provide a direct link to each platform so the user can continue their task there.
FR-10	The system shall generate a short summary of the session using an LLM API call.
FR-11	The system shall store session history (prompt metadata, results, summary) in a database.
FR-12	The system shall display results in a comparison dashboard showing token counts, cost, and recommendation side by side for all three platforms.

4. Non-Functional Requirements

Category	Requirement
Performance	Token calculation and cost estimation results should be returned within a few seconds for typical file sizes (under 10 MB).
Usability	The interface shall be simple enough for a non-technical user to upload a file and understand the results without instructions.
Reliability	The system shall handle API failures gracefully (e.g. one provider being down) and still show results for the remaining providers.
Security	API keys shall never be exposed to the frontend or committed to version control; all keys are stored as backend environment variables.
Portability	The system shall run on any modern browser and standard cloud hosting, without requiring specialized hardware.
Maintainability	Pricing and free-tier reference data shall be stored in an easily editable configuration file (JSON), not hardcoded in application logic.

5. System Modules

Module	Responsibility
Frontend (React)	Upload UI, results/comparison dashboard
File Ingestion Module	Parses PDF, DOCX, code, and images into text/metadata (PyMuPDF, python-docx, Pillow)
Token Counting Module	Calls tiktoken (OpenAI), Anthropic SDK, Google Generative AI SDK

Cost & Recommendation Engine	Applies pricing config, ranks providers, checks free-tier fit
Prompt Compression Module	Reduces prompt length/token count while preserving meaning
Chat Summarizer	Generates a short session summary via one LLM API call
Database	Stores session history, results, and summaries (SQLite/MongoDB)

Data flow: User → Frontend → Backend (File Ingestion + Token Counting run in parallel) → Cost & Recommendation Engine → Prompt Compression → Chat Summarizer → Database → Results returned to Frontend dashboard.

6. External Interface Requirements

6.1 User Interfaces

- Upload page: prompt textbox, file upload control, submit button
- Results dashboard: summary cards (tokens, files), provider comparison cards (ChatGPT, Claude, Gemini) with cost, free/paid tier info, recommended badge, and platform link; session summary panel

6.2 Software Interfaces

Interface	Purpose
OpenAI API / tiktoken	Token counting for ChatGPT (tiktoken is local/offline; API used for optional compression/summary)
Anthropic API	Token counting for Claude via count_tokens endpoint
Google Generative AI API	Token counting for Gemini via count_tokens method
Database (SQLite/MongoDB)	Persists session history and results

7. Proposed Technology Stack

Layer	Technology
Frontend	React.js, Tailwind CSS
Backend	Python, FastAPI
File Parsing	PyMuPDF/pdfplumber, python-docx, Pillow, pygments
Token Counting	tiktoken, Anthropic SDK, Google Generative AI SDK
Database	MongoDB Atlas / PostgreSQL (Supabase)
Hosting	Frontend: Vercel/Netlify; Backend: Render/Railway
Version Control	Git and GitHub

8. Known Limitations

- Free-tier message limits are not exposed via any public API and are treated as manually updated reference data, not live values.
- Image tokenization formulas differ by provider and are implemented per each provider's published documentation.
- Output token counts are estimates, since the actual AI response is not generated before the estimate is shown.
- Pricing is sourced from provider documentation and updated periodically via a configuration file, not scraped live.
- Video and audio file formats are not supported in the current scope.

9. Future Scope

- Chrome extension version that works directly inside ChatGPT/Claude/Gemini web interfaces.
- Automated periodic updating of pricing and free-tier data.
- Support for video and audio token estimation.
- A trained recommendation model replacing the initial rule-based logic.

10. Appendix

10.1 Team

- Sanket Mohite
- [Member 2 name]
- [Member 3 name]
- [Member 4 name]

10.2 References

[1] Y. Zhang, D. Zhang, B. Wang, and S. Zhang, "TokCoInfer: Token-Level Multi-Model Collaboration for Energy-Efficient LLM Inference," in Proc. IEEE ICASSP, 2026.

[2] Y. Liu, H. Zhang, Y. Miao, V. H. Le, and Z. Li, "OptLLM: Optimal Assignment of Queries to Large Language Models," in Proc. IEEE Int. Conf. Web Services (ICWS), 2024, pp. 788–798.

[3] "Problematic Tokens: Tokenizer Bias in Large Language Models," IEEE Conference Publication.

[4] "Context-Aware Autoscaling for Cost-Efficient Large Language Model Inference With Prefix Cache Integration," IEEE Journal, 2026.